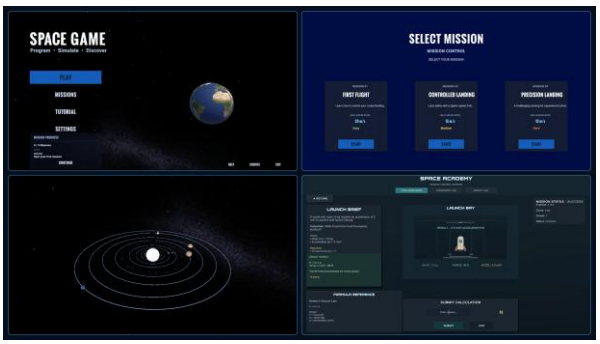
A

Project Report

On

Educational Space Game



Prepared by

Khush Vaghani (24CE135)

Shreejay Shiyani (24CE117)

Under the guidance of

Prof. Martin Parmar

Submitted to

Charotar University of Science & Technology

Degree of Bachelor of Technology

in Computer Engineering

CEUP301: Project-I

Of 5th Semester of B.Tech

Submitted at



U. & P. U PATEL DEPARTMENT OF COMPUTER ENGINEERING

Faculty of Technology & Engineering, CHARUSAT

Chandubhai S. Patel Institute of Technology

At: Changa, Dist: Anand – 388421

November 2026

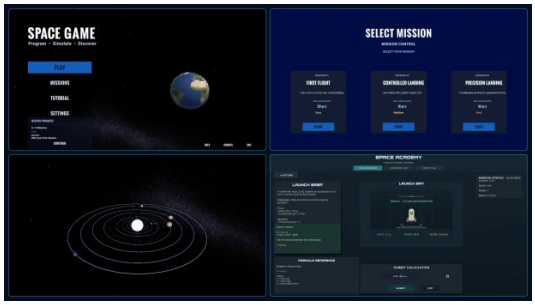
A

Project Report

On

Educational Space Game

(CEUP301 – Project-I)



Prepared by

Khush Vaghani (24CE135)

Shreejay Shiyani (24CE117)

Under the Supervision of

Prof. Martin Parmar

Submitted to

Charotar University of Science & Technology (CHARUSAT)

for the Partial Fulfillment of the Requirements for the

Degree of Bachelor of Technology (B.Tech.)

in U & P U. Patel Department of Computer Engineering (CE)

for B.Tech Semester 5

Submitted at

CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Accredited with Grade A+ by NAAC

U & P U. PATEL DEPARTMENT OF COMPUTER ENGINEERING

Chandubhai S. Patel Institute of Technology (CSPIT)

Faculty of Technology & Engineering (FTE), CHARUSAT

At: Changa, Dist: Anand, Pin: 388421.

November, 2026

CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

CERTIFICATE

This is to certify that the report entitled Educational space game is a bonafied work carried out by **khush Vaghani(24CE135) and Shreejay Shiyani(24CE117)** under the guidance and supervision of **Prof. Martin Parmar** for the subject **Project - I (CEUP301)** of 5th Semester of Bachelor of Technology in **Computer Engineering** at Faculty of Technology & Engineering (C.S.P.I.T.) – CHARUSAT, Gujarat.

To the best of my knowledge and belief, this work embodies the work of candidate himself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred to the examiner.

Under the supervision of,

Prof. Martin Parmar

U. & P. U Patel Dept. of Computer Engg.

C.S.P.I.T., CHARUSAT-Changa.

Chandubhai S Patel Institute of Technology (C.S.P.I.T.)

Faculty of Technology & Engineering, CHARUSAT

At: Changa, Ta. Petlad, Dist. Anand, PIN: 388 421. Gujarat

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled Educational Space game submitted by us to Chandubhai S. Patel Institute of Technology, Changa in partial fulfilment of the requirements for the award of the degree of **B.Tech Computer Engineering**, from U & P U. Patel Department of Computer Engineering, CSPIT, FTE, is a record of bonafide CE363 Project-III (project work) carried out by us under the guidance of **Martin Parmar**. We further declare that the work carried out and documented in this project report has not been submitted anywhere else either in part or in full and it is the original work, for the award of any other degree or diploma in this institute or any other institute or university.



(Khush Vaghani -24CE135)



(Shreejay Shiyani – 24CE117)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Dr./Prof. Martin Parmar

U & P U. Patel Department of Computer Engineering,

Chandubhai S Patel Institute of Technology (CSPIT)

Faculty of Technology (FTE)

Charotar University of Science and Technology (CHARUSAT) - Changa.

ABSTRACT

Educational Space Game is a Unity-based learning project that combines a block-programmed rocket mission with a solar-system scene and a separate Space Academy. In the mission flow, a student selects one of three landing challenges, assembles a sequence from visual programming blocks, and runs that sequence on a Rigidbody-driven rocket. A speed threshold at a configured altitude determines the displayed safe-landing or crash outcome. The space scene presents a simplified solar system and educational trajectory and orbital displays. Space Academy provides a worked force challenge, an adjustable F = m × a experiment, and a gravity comparison activity for Earth, Moon, and Mars.

The project is implemented in Unity 6000.3.2f1 for a Windows 64-bit standalone target. The inspected build settings contain five scenes: Main Menu, MissionSelect, RocketLauncher, SpaceScene, and SpaceAcademy. Mission definitions are stored as ScriptableObject assets; rocket actions are represented by an IShipBlock coroutine interface, with nested conditional and repeat blocks. The orbital content uses simplified scripted motion and an educational Earth-relative model rather than a complete n-body simulation.

The report is based on current project code, scene configuration, saved Unity screenshots, supplied planning material, and the university report-format documents. The developer team reports that functions were tested manually, and historical planning notes describe manual editor playtesting. No detailed test log, student evaluation, measured learning outcome, or automated test result was available for this review. Accordingly, the report describes implementation evidence and separates it from educational effectiveness claims.

Keywords: educational game, Unity, visual programming, Newtonian mechanics, rocket simulation, STEM learning.

ACKNOWLEDGEMENT

The project team acknowledges the guidance of Martin Parmar. The report also draws on the project's planning and implementation notes and the supplied Project-I report-format materials. The authors thank the academic evaluators for reviewing this work. Specific institutional approvals and certificate wording remain subject to the institute's review.

TABLE OF CONTENTS

ABSTRACT 7

ACKNOWLEDGEMENT 8

TABLE OF CONTENTS 9

LIST OF FIGURES 12

LIST OF TABLES 13

ABBREVIATIONS AND GLOSSARY 14

CHAPTER 1: INTRODUCTION 15

1.1 Background 15

1.2 Problem statement 15

1.3 Motivation 15

1.4 Project objectives 15

1.5 Scope 16

1.6 Audience and intended learning outcomes 16

1.7 Significance and evidence boundary 16

CHAPTER 2: EXISTING SYSTEMS AND LITERATURE REVIEW 17

2.1 Educational games and learning evidence 17

2.2 Related approaches 17

2.3 Visual programming in K–12 17

2.4 Gap and rationale 18

2.5 Literature limits 18

CHAPTER 3: REQUIREMENTS ANALYSIS 19

3.1 Stakeholders and user roles 19

3.2 Non-functional requirements 19

3.3 Hardware and software environment 20

3.4 Assumptions and constraints 20

3.5 Feasibility 21

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE 22
4.1 Scene flow 22
4.2 Runtime architecture 22
4.3 Student use cases 23
4.4 Scene inventory 24
4.5 Core script responsibilities 24
4.6 Data and persistence 25
4.7 State and error handling 25
CHAPTER 5: GAME DESIGN AND EDUCATIONAL EXPERIENCE 26
5.1 Game concept and loop 26
5.2 Missions and constraints 26
5.3 Visual programming commands 27
5.4 Solar-system exploration 27
5.5 Space Academy 28
5.6 Feedback, rewards, and usability 30
CHAPTER 6: TECHNOLOGY STACK AND IMPLEMENTATION 31
6.1 Platform and packages 31
6.2 Folder organization 31
6.3 Mission command execution 31
6.4 Mission outcome algorithm 31
6.5 Space-scene handoff and orbital model 32
6.6 Planet and Academy systems 32
6.7 Important implementation limitations 32
CHAPTER 7: TESTING AND VALIDATION 33
7.1 Evidence available 33
7.2 Test evidence record 33
7.3 Proposed test plan — not run as part of this report 33

7.4 Test limitations....34
CHAPTER 8: RESULTS AND DISCUSSION....35
8.1 Implemented outcomes....35
8.2 Objective achievement....35
8.3 Discussion....35
8.4 Evidence figures....36
CHAPTER 9: CHALLENGES, LIMITATIONS, AND FUTURE SCOPE....37
9.1 Development approach and challenges....37
9.2 Current limitations....37
9.3 Future work....37
CHAPTER 10: CONCLUSION....39
REFERENCES....40
APPENDIX A: STUDENT QUICK START....41
APPENDIX B: REQUIREMENTS AND EVIDENCE NOTES....41
APPENDIX C: ACADEMIC APPROVAL STATUS....41

In Microsoft Word, update the field to generate or refresh page numbers and section entries.

LIST OF FIGURES

Figure 4.1 Verified scene flow

Figure 4.2 Runtime architecture based on current scripts

Figure 4.3 Student use cases supported by the project

Figure 5.1 Mission selection screen

Figure 5.2 Block workspace screenshot (saved development capture)

Figure 5.3 Solar-system scene

Figure 5.4 Space Academy: challenge mode

Figure 5.5 Space Academy: force experiment

Figure 5.6 Space Academy: gravity lab

LIST OF TABLES

Table 3.1 Functional requirements and implementation status

Table 3.2 Non-functional requirements and evidence status

Table 3.3 Environment and software configuration

Table 4.1 Scene inventory

Table 4.2 Core script responsibilities

Table 5.1 Missions and constraints

Table 5.2 Block command semantics

Table 6.1 Project folder organization

Table 7.1 Evidence-led test record

Table 7.2 Proposed functional test plan

Table 8.1 Objective achievement

Table 9.1 Limitations and future work

ABBREVIATIONS AND GLOSSARY

Term	Meaning
API	Application Programming Interface
C#	C-sharp programming language
F = m × a	Newton's second law: force equals mass times acceleration
HUD	Heads-up display
IShipBlock	Project interface used by executable rocket command blocks
PlayerPrefs	Unity local key-value preference storage
Rigidbody	Unity physics component representing mass and motion
ScriptableObject	Unity asset type used to store reusable data independently of scene objects
STEM	Science, Technology, Engineering, and Mathematics
UI / UX	User interface / user experience
URP	Universal Render Pipeline
m/s	Metres per second; mission speed label used in project UI
μ	Gravitational parameter used in the simplified orbital calculations

CHAPTER 1: INTRODUCTION

1.1 Background

School learners often encounter mechanics as equations before they have a chance to experiment with how inputs affect motion. Educational games can make variables and feedback visible, but evidence from the wider field shows that benefits depend on instructional design and evaluation; a game should not be assumed to improve learning merely because it is interactive [1–3]. This project uses a space theme to connect basic physics ideas with a visual, hands-on activity.

1.2 Problem statement

The project addresses the design challenge of presenting introductory force, acceleration, velocity, gravity, and orbital ideas through an interactive computer game. The available planning notes describe a desire to let a learner reason about when to act and how much force to apply, with failure giving useful feedback. The project does not claim to solve a measured educational deficiency; no baseline study or school needs analysis was provided.

1.3 Motivation

A rocket and planetary setting provides an intuitive context for changing thrust, observing motion, and comparing gravitational environments. Visual programming blocks can expose sequence and control-flow concepts without requiring learners to write syntax. A separate Academy gives the student a place to manipulate numerical variables and see formula-driven feedback.

1.4 Project objectives

- Create a Unity game flow with mission selection and a rocket-programming activity.
- Let learners build and run an ordered sequence of visual commands, including basic control flow.
- Connect command choices to rocket movement, fuel consumption, trajectory feedback, and mission outcome.
- Provide a separate Space Academy with a force challenge, adjustable force experiment, and gravity comparison.

- Present a simplified solar-system environment with planet information and orbital/trajectory visualization.
- Support iteration through retry and local mastery progress.

1.5 Scope

The current project includes the five scenes listed in Chapter 4, three mission data assets, eight command-block kinds, a simplified orbital exploration scene, and three Academy modes. The project is a Windows 64-bit Unity application in its configured build target. No account system, teacher dashboard, learning analytics service, networking, formal classroom deployment, or school assessment integration was found. These are outside the verified implementation scope.

1.6 Audience and intended learning outcomes

The intended audience, supplied by the project team, is school students. No exact grade band, age range, curriculum alignment, or reading-level specification was provided. The activity is designed to give practice with sequencing, conditionals, repetition, force and acceleration, speed thresholds, gravity comparisons, and qualitative orbital ideas. These are intended learning opportunities, not demonstrated learning gains.

1.7 Significance and evidence boundary

The project brings together visual coding and mechanics in one game concept. Its value for student learning remains to be evaluated with learners, an appropriate comparison or pre/post measure, and teacher feedback. The current report describes functionality and educational rationale; it does not assert verified learning effectiveness.

CHAPTER 2: EXISTING SYSTEMS AND LITERATURE REVIEW

2.1 Educational games and learning evidence

Meta-analyses report positive average learning effects for serious games in some settings, while also finding variation by comparison condition, instructional support, and game design [1]. Reviews of game-based learning in primary education likewise show a growing evidence base but do not justify assuming that every game will produce learning gains [2]. Gamification research reports context-sensitive effects and highlights the importance of how game mechanics are integrated into instruction [3].

The project therefore treats immediate feedback, experimentation, and meaningful learner decisions as design mechanisms to test, not as proof of educational impact. A future classroom evaluation should measure whether learners can transfer ideas to new physics problems and should collect usability and engagement feedback separately.

2.2 Related approaches

NASA Space Place provides space-themed games and activities for school-age audiences, while NASA's interactive resources include visual exploration tools for solar-system objects and missions [4, 5]. These demonstrate that space content can support informal learning and exploration. The present project's distinguishing design direction is the combination of a block-programmed rocket challenge with a dedicated force/gravity laboratory. This is a feature-level comparison, not a claim of superiority; the products have different scope and maturity.

2.3 Visual programming in K–12

Block-based programming can lower syntax barriers and provide a visual representation of sequence and control structures. Recent synthesis work examines learning outcomes across K–12 block-based programming studies, while also showing why outcomes should be measured rather than presumed [6]. In this project, Launch, Boost, Wait, Rotate, Orbit, Stop, If, and Repeat map visible commands to code execution. The project has no supplied curriculum mapping or independent learning study.

2.4 Gap and rationale

The project's opportunity is to couple a physics-themed task with the learner-authored command sequence: students choose actions, observe motion and feedback, then revise the sequence. Space Academy offers a more direct relationship between formula inputs and displayed values. A future study should determine whether this combination improves understanding beyond exposure to the interface alone.

2.5 Literature limits

The cited sources provide general context, not evidence about this specific implementation. No user research, requirements survey, school pilot, or comparison against a named competing product was found in the accessible project materials.

CHAPTER 3: REQUIREMENTS ANALYSIS

3.1 Stakeholders and user roles

The primary role is the student player. The inspected project does not implement separate teacher, administrator, or account roles. The development team and academic supervisor are project stakeholders, not in-game roles.

ID	Functional requirement	Current evidence/status
FR-01	Navigate from main menu to mission selection and Space Academy.	Implemented in scene flow; five scenes in build settings.
FR-02	Choose among three landing missions with different speed thresholds.	Implemented using MissionData assets and MissionCardUI.
FR-03	Create, reorder, and remove commands in a visual workspace.	Implemented by BlockWorkspaceController; current scene contains block palette/workspace.
FR-04	Execute sequential commands and nested If/Repeat control flow.	Code implemented; runtime behaviour reported manually tested by developers, no test log.
FR-05	Apply propulsion, wait, rotate, orbit, stop, and fuel rules.	Implemented across block classes and RocketFuelSystem; no per-case formal results provided.
FR-06	Show trajectory and mission outcome, allow retry.	Implemented by trail, SpeedDetection, status UI and retry path.
FR-07	Explore a solar-system scene and planet details.	Scene and scripts present; saved screenshot evidence.
FR-08	Practice force, experiment with mass/acceleration, compare gravity.	Three Academy modes present; challenge is one fixed question.
FR-09	Save local Academy mastery/progress.	PlayerPrefs code present; persistence across restart not independently verified here.

3.2 Non-functional requirements

Table 3.1 Functional requirements and implementation status

ID	Quality concern	Evidence / status
NFR-01	Platform compatibility	Build target is StandaloneWindows64; packaged executable was not available for independent install verification.
NFR-02	Usability for school students	Intended audience is known; no age-band usability study or accessibility audit supplied.
NFR-03	Reliability of gameplay	Developer-reported manual testing; zero missing references in inspected scenes and zero Unity compile errors at inspection. No formal test suite

		results.
NFR-04	Educational validity	Conceptual alignment with mechanics; no student learning assessment or curriculum validation.
NFR-05	Performance	No frame-rate, memory, loading-time, or hardware benchmark available.
NFR-06	Maintainability	Features are separated into scenes/scripts and mission data assets; no code quality metrics available.
NFR-07	Accessibility/localization	No verified screen-reader support, remapping, captions, language options, or accessibility audit.

3.3 Hardware and software environment

Table 3.2 Non-functional requirements and evidence status

Item	Verified configuration
Engine	Unity 6000.3.2f1
Language	C#
Render pipeline	Universal Render Pipeline (URP)
UI	Unity UI (UGUI) and TextMeshPro
Input package	Input System 1.17.0 is included; project also emits a warning about deprecated legacy Input Manager setting
Other packages	Cinemachine 3.1.7; Splines 2.8.4; Unity Test Framework 1.6.0 (package presence is not evidence that tests were run)
Build target	Standalone Windows 64-bit
Minimum end-user hardware	Not specified or benchmarked; cannot be responsibly stated from project files

3.4 Assumptions and constraints

- A learner can operate mouse-driven UI and understand the text labels; this has not been validated for a defined age group.
- Mission speed values are presented as metres per second, while positions and physics remain Unity world units unless explicitly scaled.
- Orbital constants and scene scale are educational abstractions, not calibrated astronomical data.
- The application is configured for desktop Windows; platform support beyond the configured target is unverified.
- No formal schedule dates, cost estimate, or individual task allocation were supplied.

3.5 Feasibility

Technical feasibility is evidenced by an openable Unity project with configured scenes and scripts, and by the Unity editor reporting no compile errors during inspection. Operational feasibility for a classroom remains unknown because no packaged build, device benchmark, teacher workflow, or student study was supplied. Economic feasibility cannot be assessed because development and deployment costs were not documented.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 Scene flow

The build configuration includes Main Menu, MissionSelect, RocketLauncher, SpaceScene, and SpaceAcademy. MissionSelectManager stores the chosen MissionData in a static selection before loading RocketLauncher. LaunchAltitudeTrigger transitions a ship-tagged rocket into SpaceScene when it enters the gate. Space Academy is reachable from the menu as a separate learning path.

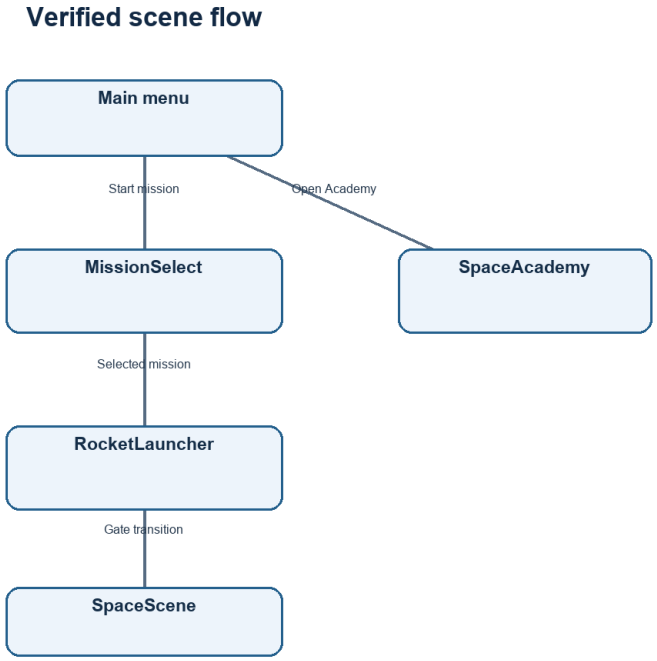


Figure 4.1 Verified scene flow

4.2 Runtime architecture

The rocket programming path is UI-driven. BlockWorkspaceController converts visible block entries into IShipBlock objects. ShipBlockRunner executes the resulting list sequentially as coroutines. Individual command classes manipulate a Rigidbody or invoke a nested sequence. RocketFuelSystem provides fuel state. SpeedDetection reads the mission data, rocket position, and velocity to determine the outcome; TrajectoryTrail records positions for visual feedback.

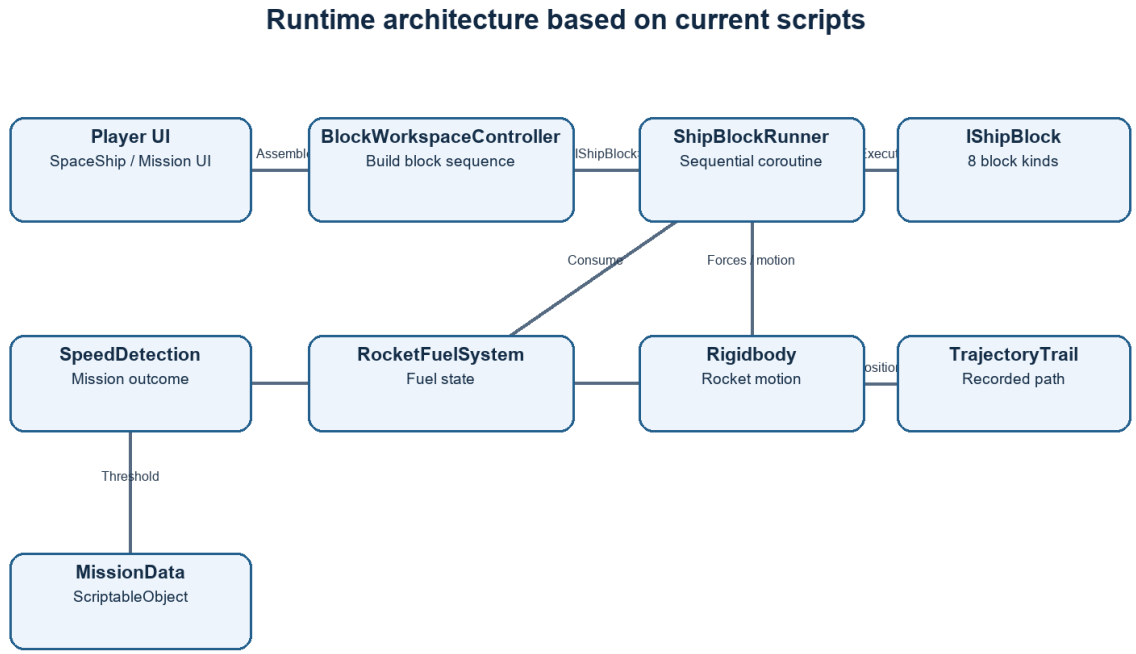


Figure 4.2 Runtime architecture based on current scripts

4.3 Student use cases

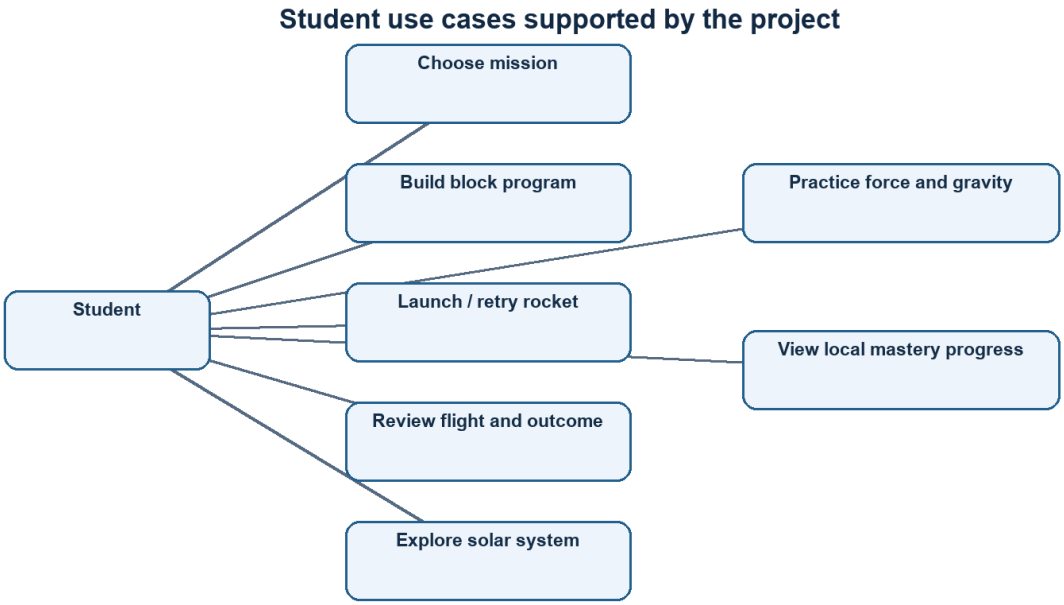


Figure 4.3 Student use cases supported by the project

4.4 Scene inventory

Table 3.3 Environment and software configuration

Scene	Main purpose	Observed core objects/components
Main Menu	Entry and navigation	MainMenuNavigation, MainMenuProgressUI, MainMenuSecondaryNavigation, settings UI
MissionSelect	Mission choice	MissionSelectManager, three MissionCardUI instances
RocketLauncher	Program and launch rocket	SpaceShip, ShipBlockRunner, RocketFuelSystem, block palette/workspace, mission/fuel/status UI, SpaceGate
SpaceScene	Solar system and flight display	PlanetOrbit components, orbit paths, EarthLandingDetector, EarthOrbitalGravity, TrajectoryPredictor/Visualizer, CruiseController
SpaceAcademy	Guided learning activities	SpaceAcademyController, AcademyModeManager, ExperimentModeController, GravityLabController, TMP text UI

4.5 Core script responsibilities

Table 4.1 Scene inventory

Script / type	Responsibility in current implementation
MissionData	Stores mission title/difficulty, threshold and target parameters as reusable asset data.
MissionSelectManager	Selects mission asset and loads RocketLauncher.
BlockWorkspaceController	Manages drag/drop, ordering, deletion, nested workspace UI, and creates command objects.
IShipBlock and block classes	Represent executable actions and control flow as coroutine-compatible commands.
ShipBlockRunner	Runs command sequence, reports progress/failure/end reason, and guards against overlapping runs.
RocketFuelSystem	Tracks fuel; Launch costs 20 and Boost costs 10 from a 100-unit starting store in the inspected configuration.
SpeedDetection	Checks downward crossing of configured altitude and compares speed with mission threshold.
SpaceRocketManager	Prepares the rocket for SpaceScene, scales and places it, assigns Earth-relative circular handoff velocity, and disables global gravity.
EarthOrbitalGravity	Applies simplified Earth-relative orbital acceleration and calculates telemetry-related state.
SpaceAcademy controllers	Provide fixed challenge, adjustable F=ma experiment, gravity comparison, tutorial and local progress UI.

4.6 Data and persistence

Mission configuration is stored in ScriptableObject assets under Assets/Mission. The selected mission is passed between scenes using a static field, so it is not a database record and is not designed for multi-session user accounts. Academy completion and selected settings use Unity PlayerPrefs, a local key-value store. Cloud synchronization, privacy controls, profiles, and teacher access are not implemented in the inspected code.

4.7 State and error handling

ShipBlockRunner tracks completion count and total count, prevents a second run from starting while one is active, and records completion, Stop block termination, out-of-fuel, or fault end states. A conditional evaluates its selected sensor and comparator when execution reaches that block; if true, it executes its nested commands. Repeat executes its nested commands a configured integer number of times. The runner's isolation behaviour should be covered by explicit tests because no detailed case-by-case results were supplied.

CHAPTER 5: GAME DESIGN AND EDUCATIONAL EXPERIENCE

5.1 Game concept and loop

The core mission loop is: choose a mission, assemble a program, start the countdown, observe rocket motion and fuel/trajectory feedback, review the mission result, and retry with a revised sequence. The Space Academy provides a separate loop of selecting a mode, changing inputs or answering a prompt, viewing feedback, and earning local completion/score progress.

5.2 Missions and constraints

Table 4.2 Core script responsibilities

Mission	Difficulty	Maximum landing speed	Target altitude	Horizontal distance
First Flight	Easy	20 m/s	70	26.07
Controlled Landing	Medium	15 m/s	70	26.07
Precision Landing	Hard	10 m/s	70	26.07

The configured mission assets share target height 70, horizontal distance 26.07, and gravity value 9.81. The outcome script checks total speed magnitude at the downward altitude crossing. These values and the threshold logic are part of the present design; they should not be interpreted as validated physical distances unless the project's world-unit scale is separately specified.

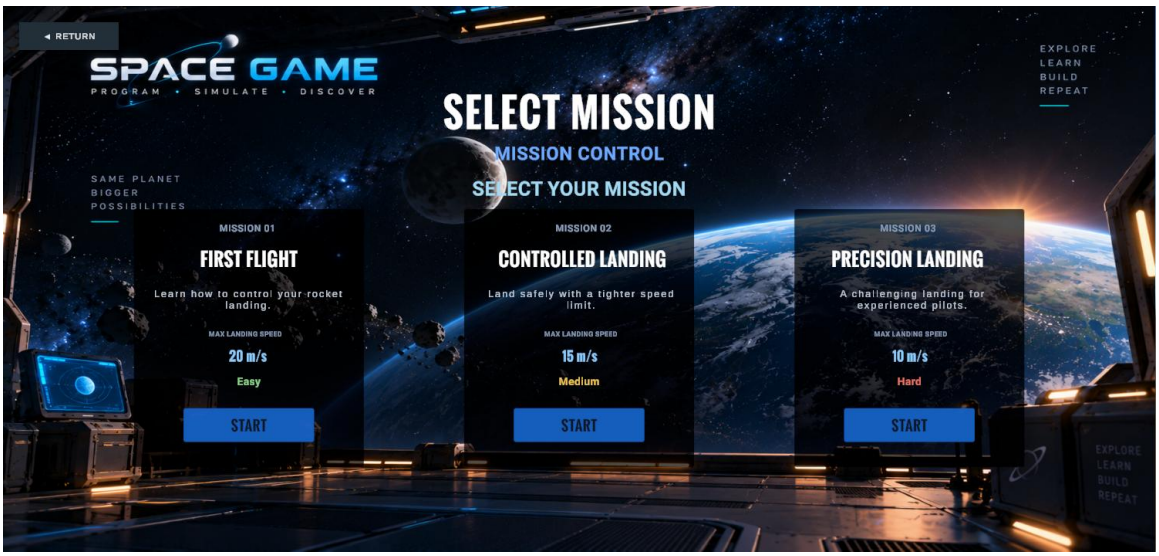


Figure 5.1 Mission selection screen (saved Unity Game View capture)

5.3 Visual programming commands

Table 5.1 Missions and constraints

Block	Observed semantics
Launch	Enables gravity and applies launch force; consumes launch fuel.
Boost	Applies force along rocket local up direction and consumes fuel.
Wait	Yields for a duration.
Rotate / Turn	Changes Rigidbody rotation and rotates velocity.
Orbit	Runs a scripted circular movement via MovePosition with gravity temporarily disabled; not a natural gravitational orbit.
Stop	Sets velocity to zero and stops the current sequence.
If	Tests altitude, speed, or vertical speed with a selected comparator; runs nested commands when true.
Repeat	Runs nested commands a configured number of times.

The screenshot below is an earlier saved development capture with six visible basic commands. The current scene file also contains If and Repeat palette entries; the image should not be read as a complete current palette inventory.

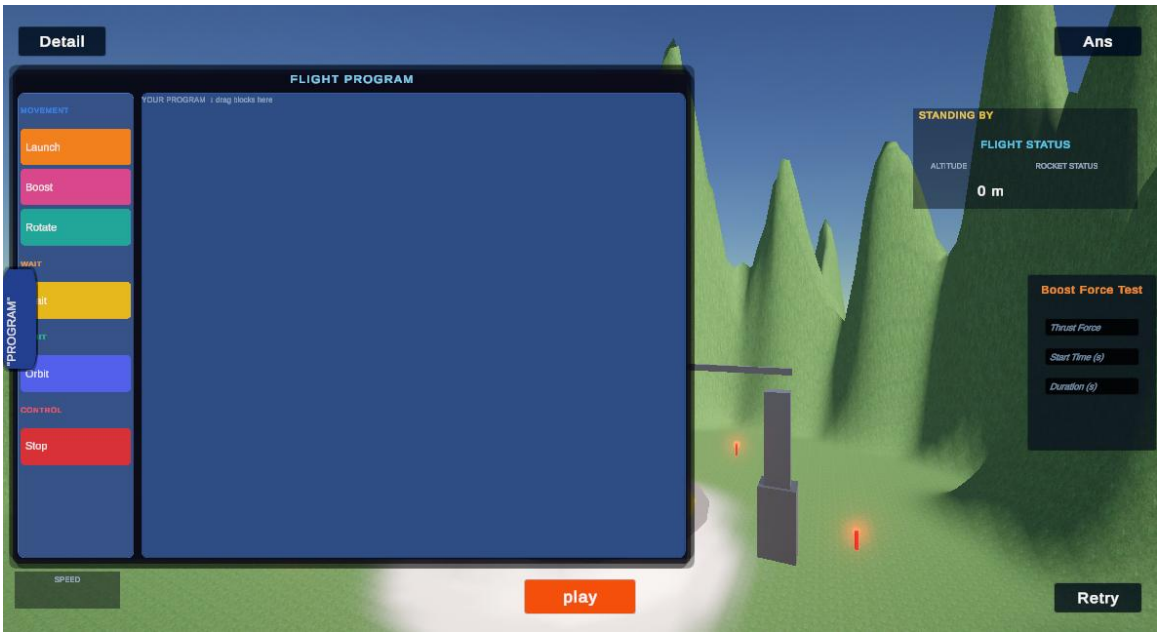


Figure 5.2 Block workspace screenshot (saved development capture; earlier six-command palette)

5.4 Solar-system exploration

The SpaceScene presents the Sun, eight planets and Moon, scripted orbital paths, an asteroid belt, stars, labels, and planet information. PlanetOrbit components animate

positions/rotation along simplified paths. EarthOrbitalGravity supplies an Earth-relative approximate gravity model; trajectory prediction and visualization show a short-horizon predicted path and markers. The system is intentionally simplified and is not a high-fidelity n-body solver.

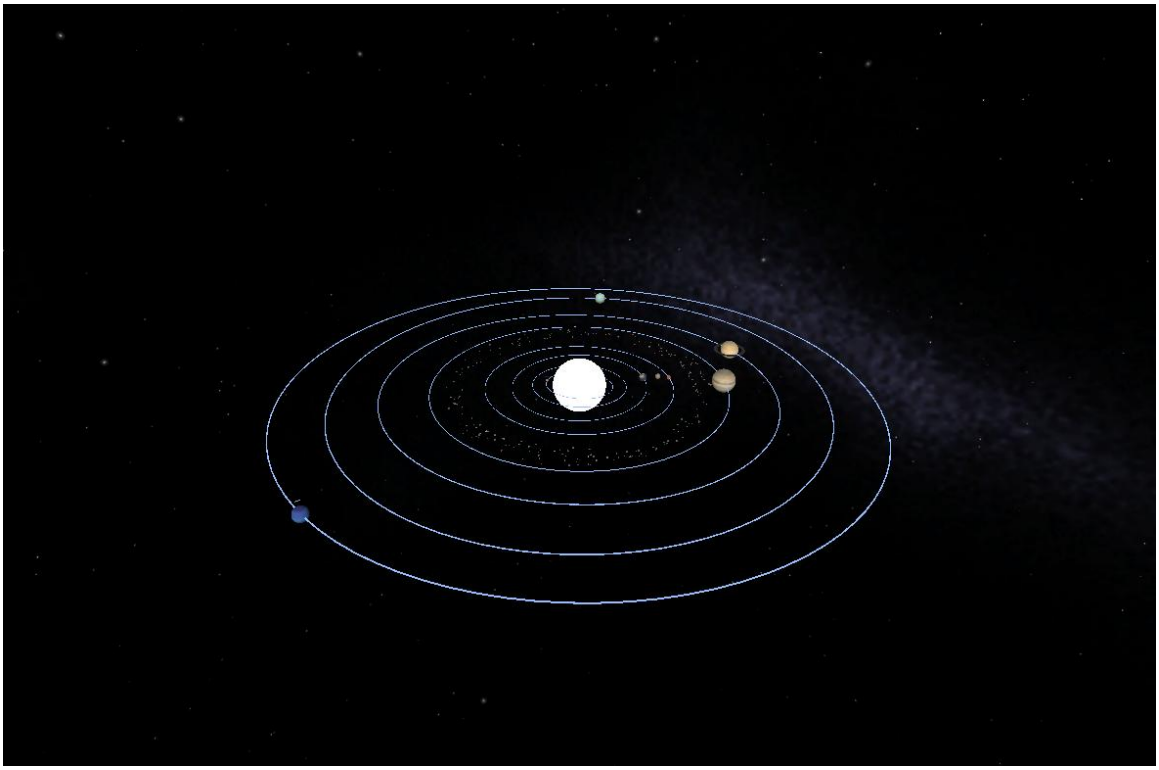


Figure 5.3 Solar-system scene (saved Unity Game View capture)

5.5 Space Academy

Challenge mode asks one fixed Newton's second law question: for mass 10 kg and acceleration 3 m/s², calculate force 30 N. The accepted value allows ±0.1 N. A hint reduces the awarded score from 10 to 5 points; streak and feedback UI are present. This is a single challenge, not a full question bank.

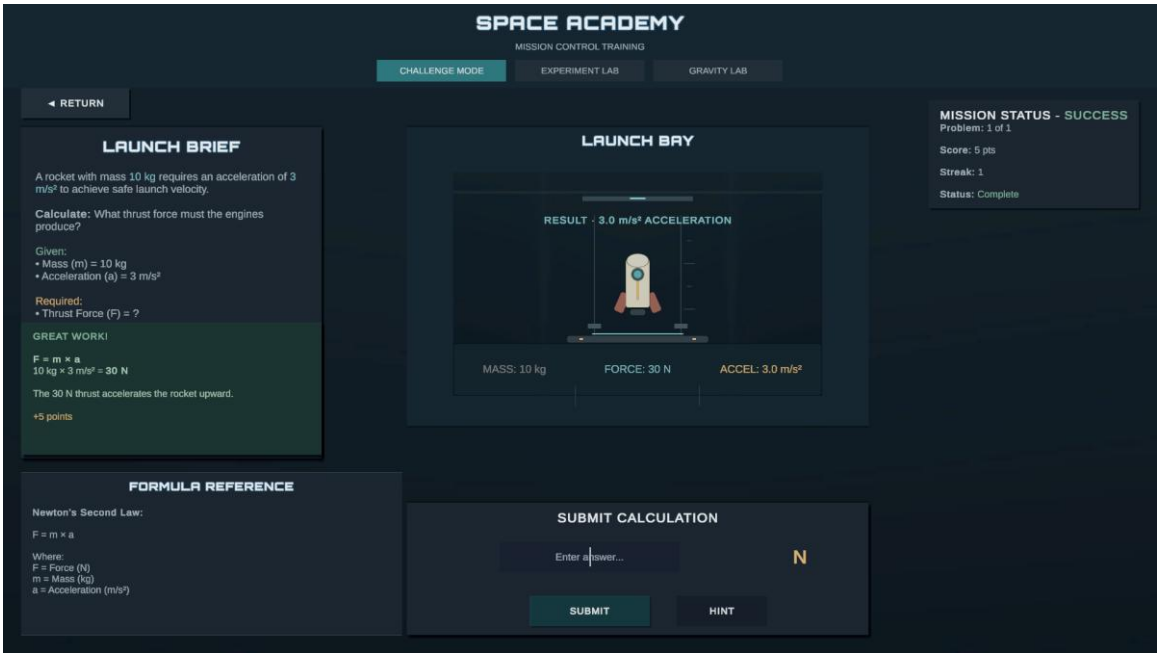


Figure 5.4 Space Academy: challenge mode (saved screen showing 30 N response)

Experiment mode exposes mass and acceleration sliders and displays F = m × a. The saved screen shows 20 kg and 4 m/s² producing 80 N, with an animated rocket. The visual response is primarily a presentation of formula inputs, not a measured rigidbody experiment.

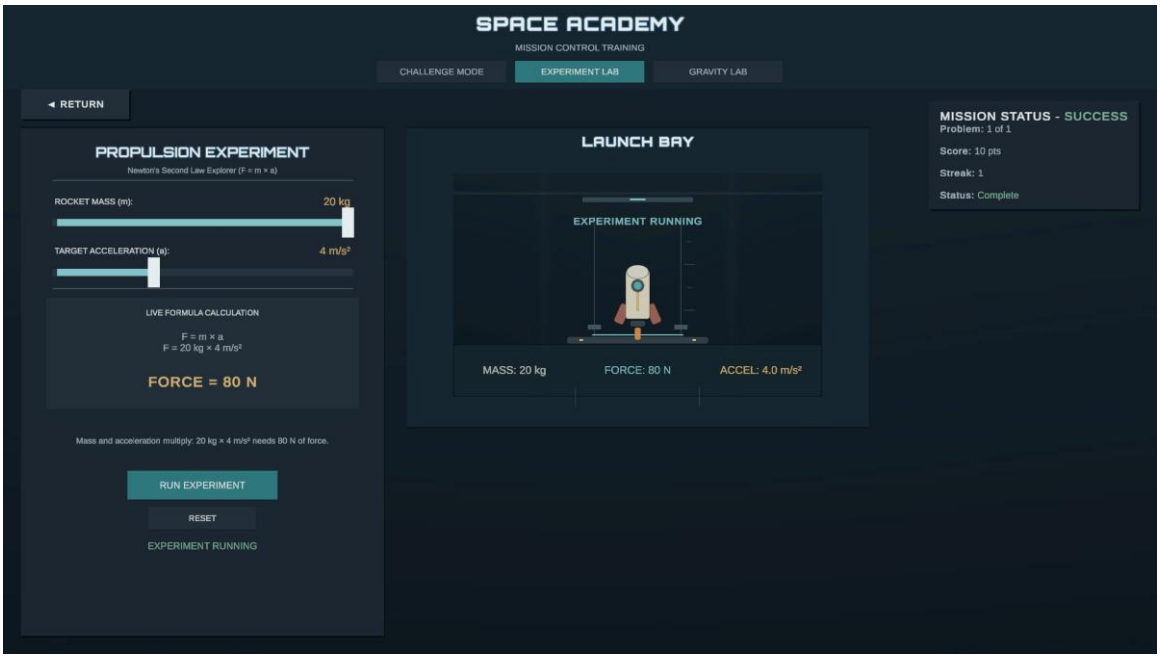


Figure 5.5 Space Academy: force experiment (saved screen showing 20 kg × 4 m/s² = 80 N)

Gravity Lab offers Earth, Moon, and Mars presets (9.81, 1.62, and 3.71 m/s²), a mass control, and the relationship W = m × g. A 120 N lift capacity is used for illustrative crate

feedback/animation. The visual is educational feedback rather than a physical load simulation.



Figure 5.6 Space Academy: gravity lab (saved screen showing Earth weight example)

5.6 Feedback, rewards, and usability

The rocket path provides countdown, fuel, block progress, speed/trajectory cues, outcome status, and retry. Academy modes provide hints, correctness feedback, score/streak, tutorial overlays, and local mastery indicators. Usability for school students, readability at different display sizes, accessibility, and learning transfer have not been independently evaluated. Saved screenshots evidence selected visual states only.

CHAPTER 6: TECHNOLOGY STACK AND IMPLEMENTATION

6.1 Platform and packages

The project is authored in Unity 6000.3.2f1 and C#. It uses URP, UGUI, TextMeshPro, Unity Rigidbody physics for the mission rocket, the Input System package, Cinemachine, and Splines. The project targets Windows 64-bit. Package presence is reported as configuration evidence, not as proof that every package is exercised in gameplay.

6.2 Folder organization

Table 5.2 Block command semantics

Path	Observed content
Assets/Scenes	The five gameplay scenes and related Unity scene files.
Assets/Scripts	Gameplay, UI, mission, block, solar-system, and Academy C# scripts.
Assets/Mission	Three mission ScriptableObject assets.
Assets/Screenshots	Saved Game View and editor captures used as visual project evidence.
Assets/UI	UI artwork and associated assets.
Packages / ProjectSettings	Unity package manifest and project configuration.

6.3 Mission command execution

The block workspace builds a List<IShipBlock>. Each block executes through an asynchronous coroutine contract. ShipBlockRunner advances through the list and publishes progress and end status. If and Repeat contain nested blocks. Launch and Boost apply Rigidbody forces; fuel is deducted from RocketFuelSystem. A 100-unit fuel store is configured, with a 20-unit launch cost and 10-unit boost cost; continuous per-second consumption is set to zero in the inspected setup.

6.4 Mission outcome algorithm

SpeedDetection monitors the rocket's vertical position relative to a configured altitude. When it detects a downward crossing, it compares the magnitude of the Rigidbody velocity with the selected mission's speed threshold. Under the threshold, it stops the rocket and disables gravity; otherwise the crash/failure path is displayed. Retry resets relevant position, velocity, fuel, landing check, and trail state. This models a speed-gated landing result rather than collision with a modeled ground surface.

6.5 Space-scene handoff and orbital model

When the gate is entered, the scene transition preserves the rocket. SpaceRocketManager scales the craft to 0.0125, positions it relative to an Earth spawn anchor, aligns its orientation, and gives it an Earth-relative circular handoff speed based on sqrt(μ/r), with μ configured as 900 and Earth's scripted orbital velocity added. Global gravity is disabled for this space scene; EarthOrbitalGravity applies its local educational acceleration. This is a simplified model with chosen units and scope, not a physically scaled solar-system simulation.

6.6 Planet and Academy systems

Planet motion is represented by scripted circular paths and transform updates. Trajectory prediction integrates a short local state estimate for visualization. In Academy, force and weight displays are formula-driven; gravity and experiment animations use transforms/visual effects rather than full physical simulations. PlayerPrefs stores selected progress locally.

6.7 Important implementation limitations

- SpaceScene Run is a resume action when paused; it does not initiate a newly authored block sequence there. Reset cancels the current block sequence and resumes time but does not reset rocket pose.
- The Orbit block is a scripted circular movement and temporarily disables gravity.
- Landing success is determined by a configured altitude crossing and speed, not collision-based touchdown.
- Solar-system paths and Earth-relative gravity use simplified, project-specific parameters.
- The input configuration produces a legacy Input Manager deprecation warning. The Input System package is installed, but migration status should be confirmed.

CHAPTER 7: TESTING AND VALIDATION

7.1 Evidence available

The project team reports that functions have been tested by the developers. The planning notes describe manual Unity Editor playtesting and iterative debugging. Saved screenshots show selected mission, Academy, and solar-system presentation states. During this review, Unity reported zero compilation errors and the five key scenes had no missing serialized references in the inspected scan. These checks do not establish that every runtime path works or that student learning outcomes are achieved.

7.2 Test evidence record

Table 6.1 Project folder organization

Evidence item	Observed/reported result	Interpretation
Developer report supplied in conversation	Team says functions were tested.	Manual testing is reported; no cases, dates, tester names, or results log provided.
Planning notes	Historical manual Editor playtesting and iteration described.	Historical process evidence; some notes predate current code and are not a current test matrix.
Unity editor inspection	Zero compile errors at inspection; deprecated Input Manager warning.	Compilation state only; not gameplay acceptance testing.
Scene reference scan	No missing serialized references found in the five key scenes.	Structural check only.
Saved screenshots	Mission selection and selected Academy/Solar System screens visible.	Evidence of those captured visual states; not proof of broad functionality.
Automated tests	Unity Test Framework package exists.	No test execution results or coverage report supplied.
Student evaluation	None supplied.	Learning effectiveness and school usability remain unvalidated.

7.3 Proposed test plan — not run as part of this report

Table 7.1 Evidence-led test record

ID	Test action	Expected result to verify	Evidence to record
T-01	Open each enabled scene from build settings.	Scene loads without missing references or console exceptions.	Unity version, console log, scene name.
T-02	Select each mission card and start.	Correct mission values reach RocketLauncher.	Mission asset, selected UI, captured values.
T-03	Build, reorder, and delete command blocks.	Workspace order matches execution order; deleted blocks do not run.	Screen recording and sequence log.

T-04	Run each basic block and both nested control-flow blocks.	Force, wait, rotation, orbit, stop, condition, and repeat follow documented semantics.	Per-case inputs, event log, resulting motion.
T-05	Exhaust fuel during Launch/Boost.	Out-of-fuel end state and UI feedback are consistent.	Fuel before/after and runner status.
T-06	Cross mission altitude at speed above/below threshold.	Outcome follows selected mission threshold and retry resets the run.	Velocity, altitude, mission, outcome.
T-07	Pause, resume, and reset in SpaceScene.	Observed state matches intended semantics; reset behaviour clarified.	State before/after and console output.
T-08	Exercise Academy formulas and persistence.	F=m×a and W=m×g outputs are correct; progress persistence is confirmed.	Input/output matrix, restart test.
T-09	Conduct school-student usability and learning pilot.	Age-appropriate usability and learning outcomes are assessed.	Consent/ethics as applicable, rubric, pre/post results.

7.4 Test limitations

The report does not invent pass/fail results for proposed tests. No formal test plan, detailed defects list, regression record, automated test report, build artifact, performance benchmark, or student study was available. Historical planning documents mention earlier interface states that have since changed; current code and scenes were treated as more authoritative.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Implemented outcomes

Inspection confirms a five-scene Unity project with a mission selector, a block-based rocket programming scene, a solar-system scene, and a three-mode Space Academy. Three mission assets define increasing speed constraints. Current rocket programming code supports eight command kinds including nested If and Repeat. The captured screenshots show the mission selector and selected Academy and solar-system presentation states.

8.2 Objective achievement

Table 7.2 Proposed functional test plan

Objective	Assessment based on available evidence
Provide mission selection and rocket program flow	Implemented in scene configuration and related scripts.
Represent block sequence and control flow	Current code and scene data include eight kinds; complete behaviour requires a documented runtime test matrix.
Connect actions with physics and feedback	Rigidbody, fuel, trajectory and outcome components are present; developer manual testing is reported without detailed logs.
Provide Space Academy concepts	Challenge, formula experiment, gravity comparison, and screenshots are present.
Explore solar-system concepts	Planet scene, orbit/trajectory components and screenshot are present; simulation is simplified.
Demonstrate learning gains	Not demonstrated; no student evaluation or learning measure supplied.

8.3 Discussion

The architecture provides a coherent educational prototype: commands are learner-authored, motion feedback can be observed, and separate activities expose elementary mechanics formulas. Its most important limitation for academic claims is lack of validation evidence. A successful scene capture or a developer's manual test does not show that a learner understands the intended concept. Future evaluation should separate software correctness, usability, engagement, and learning transfer.

8.4 Evidence figures

Figures 5.1–5.6 reproduce saved Unity Game View images from the project repository. They are historical captures, not freshly generated screenshots during this report review. Each figure is referenced in its explanatory section. No generic internet image is used as a substitute for project evidence.

CHAPTER 9: CHALLENGES, LIMITATIONS, AND FUTURE SCOPE

9.1 Development approach and challenges

The planning notes describe staged prototyping: begin with a vertical motion/landing loop, add horizontal motion and mission thresholds, add trajectory/detail feedback to reduce blind guessing, prototype force over time, and later add block programming. This is best described as iterative prototyping; the documents do not identify a formal Agile or Waterfall process. The notes also describe manual playtesting and debugging. Calendar dates, milestone acceptance criteria, and individual student task allocation were not documented.

9.2 Current limitations

Table 8.1 Objective achievement

Area	Current limitation
Educational scope	No specified grade band, curriculum alignment, teacher materials, student study, or measured learning outcomes.
Challenge content	Challenge mode contains one fixed problem; limited question variety and adaptive difficulty.
Physics fidelity	Orbital system uses scripted circular motion and a simplified Earth-relative gravity model; units/scale are not fully physical.
Landing model	Outcome is threshold crossing plus speed comparison, not a collision-based landing surface or impact model.
SpaceScene controls	Run resumes from pause; Reset does not restore rocket pose.
Validation	No detailed manual test logs, automated results, performance metrics, or packaged build verification.
Usability/accessibility	No age-specific or accessibility evaluation, localization, or alternate input audit.
Data	Player progress is local; no account, export, or teacher dashboard.

9.3 Future work

- Define grade bands, curriculum outcomes, and educator guidance with school partners.
- Expand Academy question bank and add staged difficulty, explanations, and new problem types.
- Add a real ground collider/landing model and clearly explain the difference between educational units and SI units.
- Clarify and test SpaceScene pause, resume, and reset behaviour.

- Add reproducible automated tests for mission asset selection, block semantics, fuel use, thresholds, and persistence.
- Run usability testing with the intended age group and a carefully designed learning evaluation.
- Add accessibility options such as scalable text, contrast settings, and input remapping after learner review.
- Measure Windows performance on defined hardware and provide a packaged installation build.
- Review orbital model assumptions and provide learner-facing explanations of its limits.

CHAPTER 10: CONCLUSION

Educational Space Game is an implemented Unity prototype that combines a block-programmed rocket mission, a simplified solar-system environment, and a Space Academy for introductory force and gravity ideas. The current code and scenes support a substantial set of gameplay systems, including three mission assets, eight block command types, mission feedback, orbital visualization, and local learning-mode progress.

The project objectives related to building these software features are supported by current project evidence. The available evidence does not establish that the game improves student learning, is validated for a particular school grade, or passes a formal test suite. The most responsible next step is to complete documented functional verification and then evaluate usability and learning outcomes with the intended students. Certificate and declaration pages should be appended only after the institute approves their final wording.

REFERENCES

[1] P. Wouters, C. van Nimwegen, H. van Oostendorp, and E. van der Spek, “A Meta-Analysis of the Cognitive and Motivational Effects of Serious Games,” Journal of Educational Psychology, vol. 105, no. 2, pp. 249–265, 2013. https://doi.org/10.1037/a0031311

[2] T. Hainey et al., “A systematic literature review of games-based learning empirical evidence in primary education,” Computers & Education, vol. 102, pp. 202–223, 2016. https://doi.org/10.1016/j.compedu.2016.09.001

[3] M. Sailer and L. Homner, “The Gamification of Learning: a Meta-analysis,” Educational Psychology Review, vol. 32, pp. 77–112, 2020. https://doi.org/10.1007/s10648-019-09498-w

[4] NASA, “NASA Space Place: Games.” https://spaceplace.nasa.gov/en/menu/games/ (accessed 27 Oct. 2026).

[5] NASA, “NASA Interactives.” https://www.nasa.gov/interactives/ (accessed 27 Oct. 2026).

[6] Q. Yu, K. Yu, and B. Li, “Effects of Block-Based Visual Programming on K-12 Students’ Learning Outcomes,” Journal of Educational Computing Research, published online 2024. https://doi.org/10.1177/07356331241293163

[7] Unity Technologies, “ScriptableObject,” Unity User Manual. https://docs.unity3d.com/6000.1/Documentation/Manual/class-ScriptableObject.html (accessed 27 Oct. 2026).

[8] Unity Technologies, “Rigidbody.AddForce,” Unity Scripting API. https://docs.unity3d.com/kr/current/ScriptReference/Rigidbody.AddForce.html (accessed 27 Oct. 2026).

[9] Unity Technologies, “Input System,” Unity Manual. https://docs.unity3d.com/kr/6000.0/Manual/com.unity.inputsystem.html (accessed 27 Oct. 2026).

[10] Educational Space Game project source, Unity scenes, scripts, mission assets, and screenshots, inspected 27 Oct. 2026.

[11] Educational Space Game project planning notes, “space-game-technical-doc.md,” supplied path C:\gemdev\SpaceGame, consulted 27 Oct. 2026.

[12] Project format materials supplied by the team: 1_CoverPage.doc, 2_Firstpage.doc, 3_Certificate.doc, 4_Candidate declaration.docx, and 5_Report Format-Development.doc, consulted 27 Oct. 2026.

APPENDIX A: STUDENT QUICK START

1. Open the project using Unity 6000.3.2f1 in Unity Hub. The project targets Standalone Windows 64-bit.
2. Open the Main Menu scene and run the project in the Unity Editor, or use an approved packaged Windows build if one is produced by the team.
3. Choose a mission, arrange command blocks in the workspace, and use the launch control. Observe status, fuel, trajectory, and outcome feedback; use Retry to revise the program.
4. Use the Space Academy menu to explore the force challenge, formula experiment, and gravity comparison.

This is a project-level quick-start description, not a validated end-user manual. Exact control labels and supported deployment steps should be checked against the final packaged build.

APPENDIX B: REQUIREMENTS AND EVIDENCE NOTES

The source of project claims is current Unity scenes, scripts, asset files and configuration unless an item is explicitly attributed to planning notes or the project team. Older planning notes and the supplied implementation PDF describe earlier interface or physics states; where they conflict with current source, current source was used. The saved image captures are date-stamped project evidence, not screenshots captured during a fresh Play Mode test.

Team contribution split, exact calendar timeline, supervisor designation, exact school age/grade band, minimum hardware, approved certificate wording, and formal declaration text remain unspecified. The title page contains only the supplied student and supervisor details and the Project-I format information.

APPENDIX C: ACADEMIC APPROVAL STATUS

The certificate and candidate declaration pages are included before the abstract, based on the supplied institutional templates and updated to CEUP301: Project-I, Semester 5. The supplied declaration template named CE363 Project-III, so its course reference has been adapted. All signature and approval lines are blank. Confirm the final wording and the guide designation with the department before signing or submission; these pages are not represented as approved or signed.